

JVM Performance Analyzer

User Manual

Complete guide to uploading, analyzing, and exporting JVM diagnostic artifacts.

Covers thread dumps, JFR recordings, GC logs, heap dumps, the health score engine, the diff engine, custom rules, and all export formats.

Property	Value
Document type	User Manual
Product	JVM Performance Analyzer v1.0
Date	June 2026
Audience	Developers, SRE Engineers, Performance Specialists, Engineering Managers
Languages supported	English, Spanish, French, German, Portuguese, Italian
Deployment	Self-hosted — no cloud dependency

DevManchego — Internal Documentation

Table of Contents

Table of Contents	2
1. Introduction	5
1.1 Who This Manual Is For	5
1.2 Key Capabilities at a Glance	5
1.3 Privacy & Data Security	5
2. Getting Started	7
2.1 Prerequisites	7
2.2 Running the Application	7
Option A — Maven Build + Direct Execution (Recommended for Development)	7
Option B — Docker Compose (Recommended for Team Deployments)	7
Option C — Docker Only	8
Option D — Backend Only (Skip Frontend Build)	8
2.3 Accessing the Application	8
2.4 First-Time Walkthrough	8
3. Supported File Formats	9
3.1 Format Auto-Detection	9
3.2 Multi-File Upload	9
3.3 File Size Limits	9
3.4 Generating Dump Files	9
Thread Dump (jstack)	9
GC Log (JDK 11+)	9
JFR Recording	10
4. Dashboard	11
4.1 File Upload	11
4.2 Health Score Widget	11
4.3 Metric Cards	12
4.4 Critical Diagnoses List	12
4.5 Export Buttons	13
4.6 Session History	13
5. Hotspot Panel	14
5.1 Understanding Self Time vs. Total Time	14
5.2 Table Columns	14
5.3 Layer Classification	15
5.4 Solution Inspector Drawer	15
5.5 Sorting and Filtering	16
6. Call Tree	17
6.1 How the Call Tree Is Built	17
6.2 Controls	17
6.3 Stack Trace Sanitization	18
6.4 Tree Depth Limit	18

7. Timeline Analyzer.....	19
7.1 Snapshot Table	19
7.2 Thread State Timeline Chart	19
7.3 Confirmed Deadlock Detection.....	19
7.4 Thread Swim Lanes.....	20
7.5 Hotspot Trends	21
8. Diff Engine.....	22
8.1 Assigning Sessions to Slots	22
8.2 Health Score Comparison	22
8.3 Diff Tabs	22
Hotspots	22
Diagnostics	22
Thread States	23
8.4 Delta Threshold	23
9. GC Log Analyzer.....	24
9.1 Summary Cards.....	24
9.2 Alerts.....	24
9.3 Algorithm Recommendation	24
9.4 STW Events Table.....	25
9.5 GC Log Formats Supported	25
10. Heap Dump Viewer	26
10.1 Opening the Dump in Eclipse MAT	26
11. Settings and Custom Rules	27
11.1 Language Selector	27
11.2 Theme Toggle	27
11.3 Custom Package Signatures.....	27
11.4 Configurable Thresholds	28
11.5 Ruleset Export and Import.....	28
12. Exporting Reports	30
12.1 PDF Executive Report.....	30
12.2 DOCX Report	30
12.3 JSON Raw Export	30
12.4 CSV Hotspot Export	30
12.5 Where to Find Export Buttons	30
13. Health Score Deep Dive.....	32
13.1 Scoring Formula	32
13.2 Classification	32
13.3 Dominant Factor	32
13.4 Configuring Signal Weights	32
14. Integration and Metrics.....	34
14.1 Prometheus Metrics Endpoint	34
14.2 JSON Status Endpoint.....	34

14.3	Webhook Notifications	34
14.4	Spring Boot Actuator Endpoints	35
15.	Server Configuration Reference	36
15.1	Full Configuration File.....	36
15.2	Environment Variable Overrides	36
15.3	JVM Tuning for Production	37
16.	REST API Reference	38
16.1	Endpoint Summary	38
16.2	Upload and Analyze Example (curl).....	38
16.3	Error Response Format.....	38
17.	Troubleshooting	40
17.1	Application Does Not Start	40
17.2	Upload Fails.....	40
17.3	Analysis Shows No Data	40
17.4	Export Failures	40
17.5	Browser Issues	41
18.	Appendix	42
A.	Glossary	42
B.	Keyboard Shortcuts.....	42
C.	Browser LocalStorage Keys	43
D.	Diagnostic Rule Reference	43
E.	Release Notes — Version 1.0 (June 2026)	43

1. Introduction

The JVM Performance Analyzer is a self-hosted web application designed to help software engineers, SRE teams, and performance specialists diagnose JVM runtime problems quickly and accurately. It transforms raw diagnostic artifacts — thread dumps, Java Flight Recorder (JFR) recordings, GC logs, and heap dumps — into interactive visual analyses and exportable professional reports.

All processing is performed on-premises. No data is sent to external services; your production artifacts never leave your infrastructure.

1.1 Who This Manual Is For

This manual addresses four user profiles:

- **Developer:** investigates CPU hotspots, identifies inefficient code paths, traces hotspots back to application source code.
- **SRE / Operations Engineer:** responds to production incidents, needs an at-a-glance severity signal and thread-state detail to decide on immediate action.
- **Team Lead / Engineering Manager:** reviews performance reports before a release; needs a clear summary, not raw data.
- **Performance Specialist:** compares two releases to validate that a fix actually improved the system.

1.2 Key Capabilities at a Glance

Capability	Description
Multi-format ingestion	Thread dumps, GC logs, JFR recordings, and heap dumps in a single upload
CPU hotspot analysis	Aggregated call tree with Self Time % and Total Time % for every method
Deadlock detection	Lock-graph cycle detection with monitor address tracking and confirmed-deadlock classification
GC pressure analysis	STW pause statistics, algorithm auto-detection, promotion failure and concurrent mode failure alerts
JFR profiling	Real execution sample analysis via the JDK built-in <code>jdk.jfr.consumer</code> API
Session diff	Side-by-side comparison of two analyses to detect regressions or improvements across releases
Timeline view	Multi-snapshot evolution of thread states and hotspot trends over time
Health score	Single 0-100 composite score with traffic-light classification and dominant-factor explanation
Custom rules	Define your own package signatures, layer labels, and diagnostic thresholds without redeployment
Multilingual UI	Interface available in English, Spanish, French, German, Portuguese, and Italian
Export	PDF executive report, DOCX document, JSON raw data, CSV hotspot list

1.3 Privacy & Data Security

Important

The JVM Performance Analyzer does not transmit any uploaded files or analysis results to external servers. All data is held exclusively in the JVM memory of the server process and is automatically purged after 120 minutes of inactivity. No database is used.

2. Getting Started

2.1 Prerequisites

Requirement	Details
Java	Version 21 or later (JDK, not JRE) — required to run the application
Maven	Version 3.8 or later — required to build from source
Node.js	Version 18.x — only required for frontend development (downloaded automatically by Maven during build)
Docker	Optional — required only for the Docker and Docker Compose deployment options
Browser	Chrome 120+, Firefox 120+, Edge 120+ (minimum 1280 px wide viewport)
Memory	Minimum 2 GB RAM for the JVM process; 4 GB recommended for heap dumps up to 500 MB

2.2 Running the Application

There are four ways to run the JVM Performance Analyzer, listed from simplest to most production-suitable.

Option A — Maven Build + Direct Execution (Recommended for Development)

This option builds both the backend and frontend from source, producing a single self-contained JAR file.

```
# Step 1 — Build the fat JAR (downloads Node 18 automatically)
mvn package -DskipTests

# Step 2 — Run the application
java -jar target/performance-analyzer-1.0.0.jar

# Application is available at: http://localhost:8080
```

Tip

The Maven build automatically installs Node.js 18.15.0 into the target/ directory — you do not need Node installed on your PATH for the build to succeed.

Option B — Docker Compose (Recommended for Team Deployments)

```
# Step 1 — Build the JAR
mvn package -DskipTests

# Step 2 — Build image and start container
docker-compose up --build

# Run in background
docker-compose up -d --build

# Stop the container
docker-compose down
```

Option C — Docker Only

```
mvn package -DskipTests
docker build -t performance-analyzer .
docker run -p 8080:8080 -m 4g performance-analyzer
```

Option D — Backend Only (Skip Frontend Build)

Use this when the Angular frontend is already built and you only need to iterate on backend code.

```
mvn package -DskipTests -P skip-frontend
java -jar target/performance-analyzer-1.0.0.jar
```

2.3 Accessing the Application

Once running, open your browser and navigate to:

```
http://localhost:8080
```

The application serves both the REST API (under /api/v1/) and the Angular single-page application from the same port. No separate frontend server is needed in production.

2.4 First-Time Walkthrough

The following steps describe a complete analysis from start to finish:

1. Navigate to <http://localhost:8080> — the Dashboard view opens automatically.
2. Drag and drop a thread dump file (or any supported format) onto the upload zone. Multiple files can be dropped at once.
3. The application detects the file format automatically and shows a confirmation. Click Analyze to run the full analysis pipeline.
4. The Health Score widget appears at the top of the Dashboard, showing a 0-100 score and a traffic-light indicator (green / amber / red).
5. Navigate to the Hotspot Panel (left sidebar) to see CPU-consuming methods ranked by self-time percentage.
6. Click any hotspot row to open the Solution Inspector drawer on the right, which shows the full diagnosis and recommended fix.
7. Use the export buttons (PDF, DOCX, JSON, CSV) to download a report.

3. Supported File Formats

The application accepts the following JVM diagnostic file types. You can upload multiple files from the same JVM in a single operation; they are merged into one analysis session.

Format	Extension(s)	Source Command	Detection Method
Thread Dump	.txt .log	jstack <pid> kill -3 VisualVM	Content pattern: "Full thread dump", "java.lang.Thread.State"
GC Log (Unified)	.log .txt	-Xlog:gc*:file=gc.log (JDK 9+)	Pattern: [0.000s][info][gc]
GC Log (Legacy)	.log .txt	-verbose:gc (JDK 8)	Pattern: [GC, [Full GC
JFR Recording	.jfr	jcmd <pid> JFR.start -XX:StartFlightRecording	Binary magic bytes: FLR\0
Heap Dump	.hprof	jmap -dump -XX:+HeapDumpOnOutOfMemoryError	Binary magic string: JAVA PROFILE

3.1 Format Auto-Detection

The format is determined from the file content, not the file extension. The detector reads the first 12 bytes (binary header check) and up to 20 text lines (pattern matching). This means a file named `threaddump.log` that actually contains GC log content is correctly classified as a GC log.

3.2 Multi-File Upload

When a JVM problem involves both thread contention and GC pressure, upload the thread dump and the GC log together. The system parses both files and correlates the results into a single analysis session that shows both the CPU hotspot view and the GC analyzer.

3.3 File Size Limits

The maximum allowed file size is 500 MB per file, with a total request limit of 550 MB. Files are validated client-side before upload to avoid wasted network transfers. For files larger than 2 GB (heap dumps), use Eclipse Memory Analyzer (MAT) as a standalone tool.

3.4 Generating Dump Files

Thread Dump (jstack)

```
# Find the JVM process ID
jps -l

# Capture a thread dump
jstack <pid> > threaddump.txt

# Capture 5 snapshots with 3-second intervals (for Timeline analysis)
for i in {1..5}; do jstack <pid> >> threaddump-multi.txt; sleep 3; done
```

GC Log (JDK 11+)

```
# Add to JVM startup flags:
-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=100m
```

JFR Recording

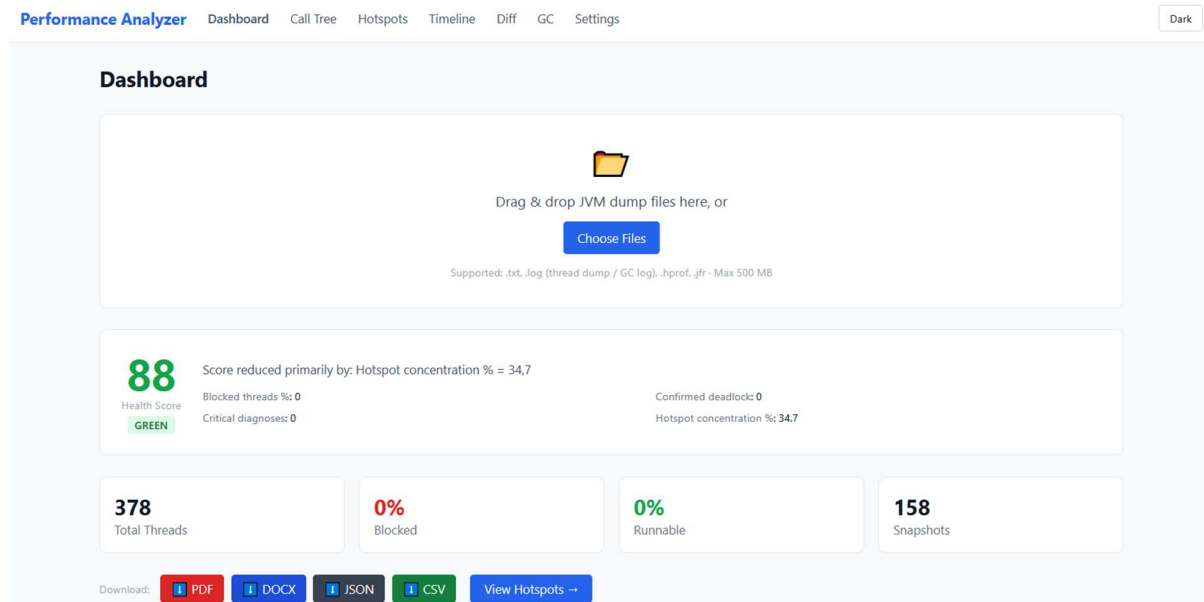
```
# Start a 60-second recording
jcmd <pid> JFR.start duration=60s filename=recording.jfr settings=profile

# Wait for recording to complete, then upload recording.jfr
```

4. Dashboard

VIEW: Dashboard — Entry point for all analysis work

The Dashboard is the home view and the starting point for every analysis. It combines the file upload interface, the health score widget, key metric cards, and session history in one screen.



4.1 File Upload

The upload zone accepts files via drag-and-drop or by clicking the area to open a file picker. Both single-file and multi-file uploads are supported in a single operation.

- Drag one or more files from your file manager and drop them onto the dashed upload area.
- Or click the upload area and select files from the dialog.
- After selection, the detected format for each file is shown as a badge (e.g., THREAD_DUMP, GC_LOG, JFR_RECORDING).
- If a file exceeds 500 MB, a red error message is shown and the file is not uploaded.

Note

Uploading does not automatically start analysis. After the upload completes successfully, click the Analyze button to run the analysis pipeline.

4.2 Health Score Widget

After analysis completes, the Health Score is displayed as a large numeric badge (0-100) with a color-coded background:

Score Range	Classification
80 - 100	GREEN — Healthy: no significant issues detected
50 - 79	AMBER — Degraded: issues present but application likely functional

Score Range	Classification
0 - 49	RED — Critical: severe issues require immediate attention

88

Health Score

GREEN

Score reduced primarily by: Hotspot concentration % = 34,7

Blocked threads %: 0

Critical diagnoses: 0

Confirmed deadlock: 0

Hotspot concentration %: 34,7

Below the score, the Dominant Factor line explains which signal penalized the score the most. Example: "Score reduced primarily by: 38% of threads are BLOCKED".

The Signal Breakdown section shows the four contributing signals and their individual scores:

- Blocked Threads (weight 30%): penalized when more than 0% of threads are BLOCKED; reaches 0 pts at 50% blocked.
- Deadlock (weight 25%): 100 pts if no confirmed deadlock; 0 pts if any deadlock is confirmed.
- Critical Diagnoses (weight 25%): penalized per CRITICAL-severity diagnosis; reaches 0 pts at 5 or more CRITICAL diagnoses.
- Hotspot Concentration (weight 20%): penalized when the top hotspot method consumes more than 10% self-time; reaches 0 pts at 50%.

4.3 Metric Cards

Four stat cards provide an instant summary of the most important thread-state metrics:

Card	Meaning
Total Threads	Total number of threads across all snapshots in the dump
Blocked %	Percentage of threads in BLOCKED state (waiting to acquire a monitor)
Runnable %	Percentage of threads in RUNNABLE state (actively using CPU or ready to)
GC P99 Pause	99th-percentile stop-the-world pause duration in milliseconds (shown only when a GC log was uploaded)

378

Total Threads

0%

Blocked

0%

Runnable

158

Snapshots

4.4 Critical Diagnoses List

The section below the metric cards lists all CRITICAL and WARNING diagnoses generated by the rule engine. Each entry shows the severity badge, the diagnosis message, and the affected method or monitor address (where applicable).

4.5 Export Buttons

The Dashboard provides direct download links for the active session:

- PDF — Server-generated executive report (see Chapter 11 for full content description).
- DOCX — Microsoft Word document with the same content as the PDF.
- JSON — Complete raw analysis result in machine-readable format.
- CSV — Flat hotspot list suitable for import into spreadsheet tools.



4.6 Session History

The Recent Sessions list at the bottom of the Dashboard shows all uploads made during the current browser session. Clicking an entry reloads its analysis result without re-running the pipeline. Sessions are listed with their file names, dump type, analysis timestamp, and health score.

Important

Sessions are stored in server memory and are automatically evicted after 120 minutes of inactivity. Closing the browser tab does not delete sessions — they persist on the server until evicted. To delete a session immediately, use the trash icon in the session history list.



5. Hotspot Panel

VIEW: Hotspot Panel — CPU method ranking by self-time

The Hotspot Panel ranks all methods that appeared at the top of at least one thread stack by their Self Time percentage. This is the primary view for diagnosing CPU bottlenecks.

Performance Analyzer Dashboard Call Tree **Hotspots** Timeline Diff GC Settings Dark

Hotspot Panel Download: PDF DOCX CSV

#	METHOD	LAYER	SELF%	TOTAL%	SEVERITY
1	..misc.Unsafe.park()	JDK Core	34.7%	34.7%	WARNING
2	..misc.Unsafe.park()	JDK Core	12.2%	12.2%	WARNING
3	..misc.Unsafe.park()	JDK Core	3.2%	3.2%	WARNING
4	..misc.Unsafe.park()	JDK Core	2.9%	2.9%	WARNING
5	..lang.Object.wait0()	JDK Core	2.9%	2.9%	WARNING
6	..engine.GameState.isLegalMove()	Business Logic	2.1%	3.4%	
7	..engine.GameState.isLegalMove()	Business Logic	2.1%	3.7%	
8	..engine.GameState.isKingInCheck()	Business Logic	1.9%	1.9%	

5.1 Understanding Self Time vs. Total Time

Metric	Definition and Use
Self Time %	Percentage of stack samples where this method was the leaf (top of stack). A high Self Time % means this method itself is consuming CPU — it is the actual bottleneck.
Total Time %	Percentage of stack samples where this method appeared anywhere in the call stack (including as a caller). Useful for understanding which callers are responsible for driving execution into a hotspot.

Example

If `OrderService.processAll()` calls `hibernate.Session.flush()` and `flush()` always appears at the top of the stack, then: `flush()` has high Self Time %, while `processAll()` has high Total Time %. The bottleneck is `flush()`, triggered by `processAll()`.

5.2 Table Columns

Column	Description
#	Rank by Self Time % (1 = highest CPU consumer)
Method	Shortened method signature. Hover for the full qualified name.
Layer	Detected ecosystem layer (see Section 5.3). Helps distinguish application code from framework or JDK code.
Self %	Self Time percentage, heat-colored: red $\geq 20\%$, orange $\geq 10\%$, amber $\geq 5\%$, neutral $< 5\%$

Column	Description
Total %	Total Time percentage
Severity	Diagnosis severity badge: CRITICAL (red), WARNING (amber), INFO (blue), or none

5.3 Layer Classification

Every hotspot method is automatically classified into one of the following ecosystem layers based on its package prefix:

Layer	Example Packages
Database / Network	org.postgresql, com.mysql, oracle.jdbc, sun.nio.ch
ORM	org.hibernate, jakarta.persistence, org.springframework.orm
JSON	com.fasterxml.jackson, com.google.gson, org.json
Cryptography	javax.crypto, sun.security, org.bouncycastle
HTTP / Web	org.apache.tomcat, io.undertow, org.springframework.web
JDK Core	java.lang, java.util, java.io, sun.*
Business Logic	Your application's own packages (configurable in Settings)

5.4 Solution Inspector Drawer

Clicking any row in the hotspot table opens the Solution Inspector — a panel that slides in from the right side of the screen. It shows:

- Full qualified method signature
- Self Time % and Total Time % values
- Detected layer category
- Diagnosis message (if the rule engine matched a known anti-pattern)
- Severity level with colored badge
- Top caller method — the first frame above this hotspot in the call tree
- Rule source label: Built-in rule or Custom rule (user-defined)

Performance Analyzer Dashboard Call Tree Hotspots Timeline Diff GC Settings Dark

Hotspot Panel

#	METHOD	LAYER	SELF%	TOTAL%	SEVERITY
1	..misc.Unsafe.park()	JDK Core	34.7%	34.7%	WARNING
2	..misc.Unsafe.park()	JDK Core	12.2%	12.2%	WARNING
3	..misc.Unsafe.park()	JDK Core	3.2%	3.2%	WARNING
4	..misc.Unsafe.park()	JDK Core	2.9%	2.9%	WARNING
5	..lang.Object.wait0()	JDK Core	2.9%	2.9%	WARNING
6	..engine.GameState.isLegalMove()	Business Logic	2.1%	3.4%	
7	..engine.GameState.isLegalMove()	Business Logic	2.1%	3.7%	
8	..engine.GameState.isKingInCheck()	Business Logic	1.9%	1.9%	
9	..engine.GameState.isLegalMove()	Business Logic	1.6%	2.4%	
10	..engine.GameState.isKingInCheck()	Business Logic	1.6%	1.6%	
11	..engine.GameState.isKingInCheck()	Business Logic	1.3%	1.3%	

Download: PDF DOCX CSV

Solution Inspector

Method
jdk.internal.misc.Unsafe.park()

Layer
JDK Core

Self Time Total Time
2.9% 2.9%

WARNING Built-in
High time in JDK internals — trace back to calling application code.

Top Caller
java.util.concurrent.locks.LockSupport.park()

5.5 Sorting and Filtering

Click any column header to sort the table ascending or descending. The table remembers the sort preference during the session. All diagnoses generated by the rule engine (including those not tied to a specific hotspot) are listed in the All Diagnoses section below the table.

6. Call Tree

VIEW: Call Tree — Aggregated hierarchical view of all stack traces

The Call Tree view displays a merged, hierarchical representation of all thread stack traces captured in the dump. Where multiple threads share identical call paths, the paths are collapsed into a single tree node whose sample counter reflects how many threads followed that path.

Performance Analyzer Dashboard Call Tree Hotspots Timeline Diff GC Settings Dark

Call Tree Inverted Tree PDF DOCX

```

graph TD
    A["_image.ImageFetcher.run() total=0.8% JDK Core"] --> B["_image.ImageFetcher.fetchloop() total=0.8% JDK Core"]
    B --> C["_image.ImageFetcher.nextImage() total=0.8% JDK Core"]
    C --> D["_lang.Object.wait() total=0.8% JDK Core"]
    D --> E["_lang.Object.wait@() total=0.8% self=0.8% JDK Core"]
    E --> F["_awt.EventQueue.dispatchEvent() total=5.8% Business Logic"]
    F --> G["_security.AccessController.doPrivileged() total=5.8% Business Logic"]
    G --> H["_security.AccessController.executePrivileged() total=5.8% Business Logic"]
    H --> I["_awt.EventQueue.run() total=0.5% Business Logic"]
    I --> J["_awt.EventQueue$4.run() total=0.5% Business Logic"]
    J --> K["_awt.EventQueue.dispatchEventImpl() total=0.5% Business Logic"]
    K --> L["_event.InvocationEvent.dispatch() total=0.5% Business Logic"]
    L --> M["_swing.RepaintManager$ProcessingRunnable.run() total=0.5% Business Logic"]
    M --> N["_swing.RepaintManager.prePaintDirtyRegions() total=0.5% Business Logic"]
  
```

6.1 How the Call Tree Is Built

For every snapshot in the dump, each thread contributes its stack trace from the outermost frame (entry point) down to the innermost frame (top of stack). When two threads share the first N frames, those frames are merged into shared nodes in the tree. The result is a tree where:

- Root nodes represent thread entry points (java.lang.Thread.run, Runnable.run, etc.)
- Leaf nodes are the innermost frames — the actual running code
- Each node's Sample Count shows how many individual thread samples flowed through it

6.2 Controls

Control	Action
Search box	Type any part of a method name to highlight and auto-expand matching nodes. Non-matching branches are grayed out but remain visible.
Inverted toggle	Switches between top-down mode (entry points at root) and bottom-up mode (hotspot methods at root). Inverted mode is useful for finding which entry points share the same hot leaf.
Node expand / collapse	Click any node chevron to expand or collapse its children. The tree remembers expansion state during the session.
Export (PDF / DOCX)	Downloads an export report for the active session directly from the Call Tree view.

6.3 Stack Trace Sanitization

The application automatically removes noise introduced by Spring AOP proxies, CGLIB-generated classes, Hibernate interceptors, and reflective lambda invocations. These synthetic frames are collapsed into a single annotated placeholder so the tree reflects actual application logic depth, not framework infrastructure.

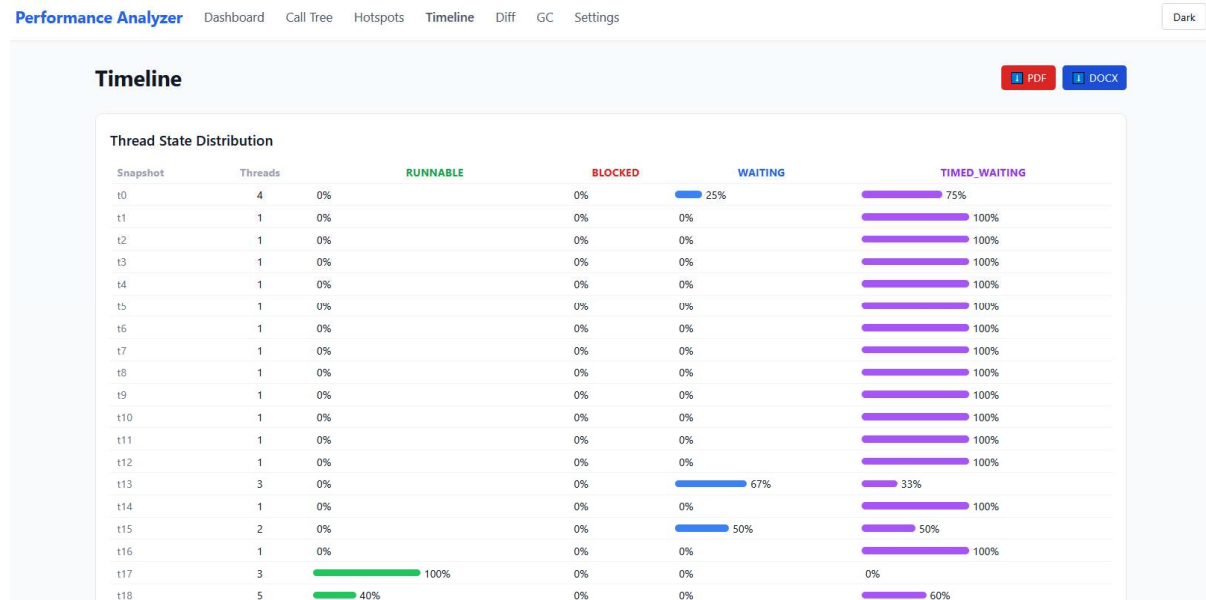
6.4 Tree Depth Limit

The call tree serializer enforces a maximum depth of 200 levels to prevent excessive memory use and rendering slowness for pathological stack traces. If a stack trace is deeper than 200 frames, the tree is truncated at that depth and a warning is displayed in the UI.

7. Timeline Analyzer

VIEW: Timeline — Multi-snapshot thread state evolution over time

The Timeline view is available when a thread dump file contains multiple sequential snapshots (e.g., generated by running `jstack` several times in a row and concatenating the output). It reveals whether performance problems are transient or sustained.



7.1 Snapshot Table

The top section lists each snapshot detected in the dump. Each row shows:

- Snapshot index (t0, t1, t2, ...)
- Timestamp (if present in the `jstack` output)
- Total thread count
- Count and percentage of RUNNABLE, BLOCKED, WAITING, and TIMED_WAITING threads
- An inline mini bar chart for the state distribution

7.2 Thread State Timeline Chart

Below the table, a line chart plots the percentage of threads in each state across snapshots. Multiple series are plotted simultaneously — one per state. Interpretation guide:

- A BLOCKED spike that returns to baseline: transient lock contention, likely caused by a short-lived operation.
- A permanently elevated BLOCKED line: sustained deadlock or a long-held monitor.
- A sudden drop in RUNNABLE threads: GC stop-the-world event, external I/O wait, or a batch job completing.

7.3 Confirmed Deadlock Detection

If a thread remains BLOCKED on the same monitor address across every snapshot in the dump, the application classifies it as a Confirmed Deadlock (rather than a suspected one). A persistent red CONFIRMED DEADLOCK badge is displayed at the top of the Timeline view. The affected thread and monitor address are shown in the badge tooltip.

Tip

To obtain confirmed deadlock evidence, capture at least 3 jstack snapshots separated by 5-10 seconds while the application is under load. If the same threads remain BLOCKED on the same monitor across all snapshots, the deadlock is confirmed.

7.4 Thread Swim Lanes

The Thread Swim Lanes section shows a color-coded per-thread state timeline for the first 20 threads. Each row is one thread; each column is one snapshot. Cell color indicates state:

Color	Thread State
Green	RUNNABLE
Red	BLOCKED
Yellow	WAITING
Purple	TIMED_WAITING
Gray	NEW or TERMINATED



7.5 Hotspot Trends

The Hotspot Trends grid shows the top CPU-consuming methods and how their Self Time % changed snapshot by snapshot. An upward arrow indicates a method is consuming more CPU over time; a downward arrow indicates improvement. This is particularly useful for detecting scheduled jobs: a method that spikes only in snapshot t3 likely corresponds to a cron task firing at that moment.

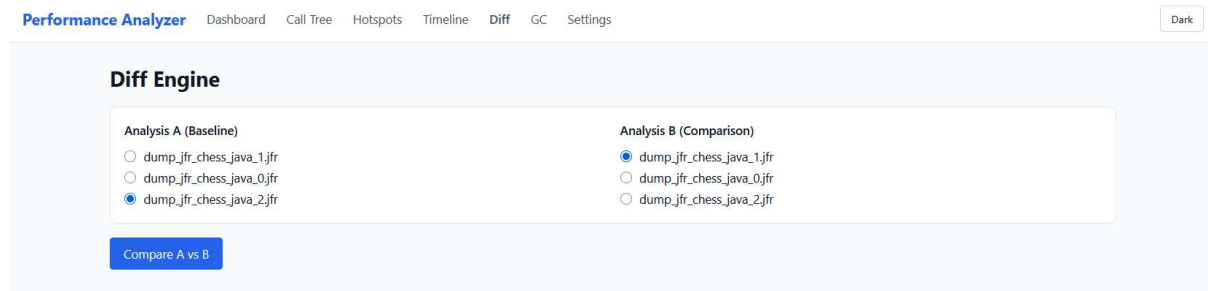


8. Diff Engine

VIEW: Diff — Side-by-side comparison of two analysis sessions

The Diff Engine compares two analysis sessions to identify regressions or improvements between builds, environments, or time periods. Typical use cases:

- Comparing a before-fix dump to an after-fix dump to validate that a performance fix worked.
- Comparing production to staging to understand environment-specific differences.
- Comparing two different time periods during a single incident to track degradation.



8.1 Assigning Sessions to Slots

The Diff view shows two groups of radio buttons labeled Analysis A (Baseline) and Analysis B (Comparison). Any session that has been analyzed can be assigned to a slot:

8. Upload and analyze the first set of files (this becomes the candidate for slot A).
9. Use the radio buttons in the Diff view to assign it to slot A.
10. Upload and analyze the second set of files.
11. Assign the second session to slot B.
12. Click Compare A vs B to compute the diff.

8.2 Health Score Comparison

At the top of the diff result, the Health Scores for session A and session B are shown side by side. A colored delta badge shows the change: green for improvement, red for regression. Example: +12 points (green) means the second build is healthier.

8.3 Diff Tabs

Hotspots

A unified table showing every method that appeared as a hotspot in either session. Columns:

- Method — shortened signature (hover for full name)
- Rank A — rank in session A (empty if not present)
- Rank B — rank in session B (empty if not present)
- Self% A and Self% B — self-time percentage in each session
- Delta — change in self-time (red if increased, green if decreased)
- Trend badge: NEW (appeared only in B), RESOLVED (disappeared in B), UP (worsened), DOWN (improved), STABLE

Diagnostics

Three columns of diagnostic information:

- Resolved (green column): diagnoses present in A but not in B — problems that were fixed.

- New (red column): diagnoses present in B but not in A — regressions introduced.
- Persisting (gray column): diagnoses present in both — ongoing issues.

Thread States

Two side-by-side horizontal bar charts showing the thread state distribution for session A and session B. Useful for comparing overall thread health between builds.

8.4 Delta Threshold

The diff engine uses a configurable delta threshold (default 5 percentage points) to classify a self-time change as significant. Changes smaller than the threshold are reported as STABLE. The threshold can be changed in the Settings view under the `hotspotDeltaThreshold` option.

9. GC Log Analyzer

VIEW: GC Analyzer — Garbage collection behavior analysis

The GC Log Analyzer view is available when one or more GC log files are included in the upload. It surfaces garbage collection behavior including pause times, algorithm detection, and common GC problems.

9.1 Summary Cards

Card	Description
GC Algorithm	The detected garbage collector: G1GC, ZGC, Shenandoah, ParallelGC, SerialGC, or CMS. The color of the card reflects algorithm modernity (red = deprecated CMS, green = modern ZGC/Shenandoah).
GC Time %	Percentage of total elapsed time spent in GC pauses. Values above 10% trigger a GC Thrashing warning.
Max Pause	Longest single stop-the-world (STW) event in milliseconds.
P99 Pause	99th-percentile pause duration. More representative than max for SLA assessment.

9.2 Alerts

The GC Analyzer displays alert banners when critical GC problems are detected:

Alert	Meaning and Recommended Action
Promotion Failure	Objects from Young Generation could not be promoted to Old Generation because Old Gen is full. Immediate risk of OutOfMemoryError. Action: increase heap size (-Xmx) or investigate object retention.
Concurrent Mode Failure (CMS)	CMS concurrent collection could not finish before Old Gen became full, forcing a stop-the-world Full GC. Action: migrate to G1GC or ZGC.
GC Thrashing (> 10%)	More than 10% of elapsed time is spent in GC pauses. Application throughput is significantly degraded. Action: increase heap or reduce allocation rate.

9.3 Algorithm Recommendation

The recommendation box below the alerts provides algorithm-specific guidance based on what was detected in the log:

Algorithm Detected	Recommendation Shown
PARALLEL	Good throughput but high pause times. For latency-sensitive workloads or heaps > 4 GB, migrate to G1 (-XX:+UseG1GC).
SERIAL	Single-threaded GC. Recommended only for heaps under 256 MB or single-CPU environments.

Algorithm Detected	Recommendation Shown
G1	Good general-purpose choice. Tune -XX:MaxGCPauseMillis to your latency SLA.
ZGC	Sub-millisecond pauses. Works best with heaps >= 1 GB.
SHENANDOAH	Very low pause times. Best for moderate heap sizes (4-32 GB).
CMS	Deprecated since JDK 9. Migrate to G1 or ZGC immediately.

9.4 STW Events Table

The events table lists every individual stop-the-world event detected in the log. It is scrollable and contains:

- Timestamp (seconds since JVM start)
- GC event type (Young, Mixed, Full, Remark, Cleanup, etc.)
- Pause duration in milliseconds (color-coded: red >= 500 ms, amber >= 100 ms)
- Heap occupancy before and after the event in MB

Tip

Sort the STW Events table by Pause to find the worst single pause event. Compare the heap before/after values to assess how effective each collection was. If heap after is still above 80%, Old Gen is under pressure.

9.5 GC Log Formats Supported

Format	Example Log Line
Unified (JDK 9+)	[0.543s][info][gc] GC(12) Pause Young (Normal) (G1 Evacuation Pause) 512M->256M(1024M) 12.500ms
Legacy (JDK 8)	0.543: [GC (Allocation Failure) [PSYoungGen: 524288K->65536K(614400K)] 524288K->131072K(1638400K), 0.0125000 secs]

10. Heap Dump Viewer

VIEW: Heap Dump — Memory retention and leak analysis

Heap dump analysis requires the Eclipse Memory Analyzer (MAT) API library, which is not published on Maven Central and therefore cannot be bundled in the standard distribution JAR.

Current Status

In the current release (v1.0), the Heap Dump view displays an informational message when a .hprof file is uploaded. The application stores the raw bytes of the dump for the duration of the session (up to 500 MB), and the view provides instructions for loading the file in standalone Eclipse MAT.

The Heap Dump view will support the following analyses in a future release:

- Dominator tree: for each significant object, the immediate dominator (the single object whose removal would make it unreachable).
- Class histogram: all classes sorted by instance count and total shallow size.
- Memory leak candidates: objects retained via static fields, uncleaned ThreadLocals, and unreleased event listeners.

10.1 Opening the Dump in Eclipse MAT

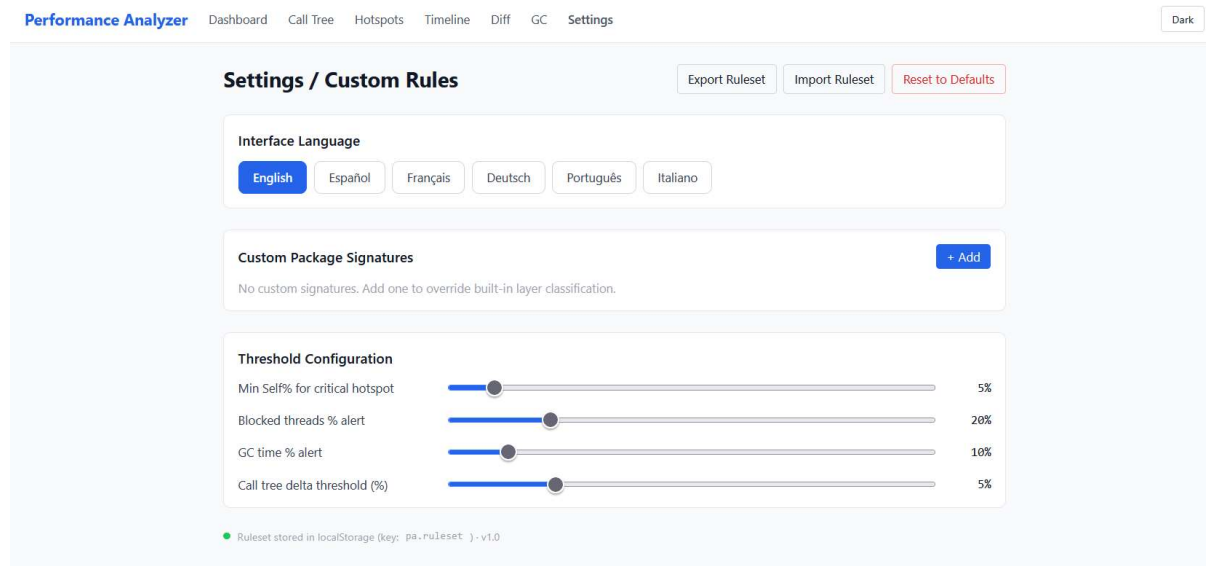
While heap analysis is not yet built in, the .hprof file can be analyzed in standalone Eclipse Memory Analyzer. Steps:

13. Download Eclipse MAT from eclipse.dev/mat
14. Start Eclipse MAT and select File > Open Heap Dump
15. Select the .hprof file you uploaded to the Performance Analyzer
16. Use the Leak Suspects report for an automated analysis of the top memory retention paths

11. Settings and Custom Rules

VIEW: Settings — Language, custom signatures, and threshold configuration

The Settings view allows users to customize the analysis rule engine, the interface language, and the display theme — all without modifying server configuration or redeploying the application. All settings are persisted in the browser's localStorage and survive tab closure.



11.1 Language Selector

The UI language can be changed at any time without page reload. Six languages are available:

Language	Notes
English (EN)	Default language. All feature documentation is in English.
Spanish (ES)	Español
French (FR)	Français
German (DE)	Deutsch
Portuguese (PT)	Português
Italian (IT)	Italiano

Click the desired language button in the Language section of the Settings view. The preference is saved under the localStorage key `pa.lang` and restored on the next visit.

11.2 Theme Toggle

The dark / light mode toggle is accessible from the top navigation bar (moon / sun icon). The preference is saved under the localStorage key `pa.theme`. Dark mode uses a deep navy color palette optimized for extended analysis sessions in low-light environments.

11.3 Custom Package Signatures

The signature dictionary maps package prefixes to layer categories and optional diagnosis messages. You can add your own entries to ensure that your application's packages are correctly labeled in the Hotspot Panel and Solution Inspector.

To add a custom signature:

17. Click Add Signature in the Custom Signatures section.
18. Enter the package prefix (e.g., com.mycompany.payments).
19. Select a layer category from the dropdown, or type a custom label.
20. Optionally add a custom diagnosis message (shown in the Solution Inspector when this package is a hotspot).
21. Click Save. The signature takes effect on the next analysis run.

Priority

Custom signatures always take priority over built-in entries. If your package com.mycompany matches a built-in signature AND a custom one, the custom one wins. The Solution Inspector shows the rule source: Built-in rule or Custom rule (user-defined).

11.4 Configurable Thresholds

Threshold	Default	Effect
blockedThreadsWarningPercent	20%	Percentage of BLOCKED threads above which a WARNING contention diagnosis is fired
gcThrashingPercent	10%	GC time % above which a GC Thrashing warning is fired
hotspotDeltaThreshold	5 pts	Minimum self-time change for a diff entry to be classified UP/DOWN

11.5 Ruleset Export and Import

All custom signatures and threshold overrides can be exported as a portable JSON file and shared with teammates so that the entire team applies consistent analysis rules:

- Export: click Export Ruleset to download a ruleset.json file containing all current custom configuration.
- Import: click Import Ruleset and select a previously exported ruleset.json. The current settings are replaced.
- Reset: click Reset to Defaults to clear all custom signatures and restore all thresholds to factory values. This action cannot be undone.

The exported JSON structure:

```
{
  "version": "1.0",
  "exportedAt": "2026-06-14T10:00:00Z",
  "customSignatures": [
    {
      "prefix": "com.mycompany.payments",
      "layer": "Payment Processing",
      "diagnosis": "Payment service hotspot - check DB connection pool size",
      "severity": "WARNING"
    }
  ],
  "thresholds": {
    "blockedThreadsWarningPercent": 15,
```

```
"gcThrashingPercent": 8,  
"hotspotDeltaThreshold": 3  
}  
}
```

12. Exporting Reports

The JVM Performance Analyzer supports four export formats. All exports are generated server-side and downloaded directly by the browser without any third-party cloud dependency.

12.1 PDF Executive Report

The PDF report is a professionally formatted document suitable for sharing with management or attaching to incident tickets. It is generated by the iText 8 library on the server.

Content:

- Cover section: analysis metadata (file names, dump types, analysis timestamp)
- Health Score: numeric score, classification, and dominant factor explanation
- Top 10 Hotspots table: method, layer, Self%, Total%, diagnosis severity
- Diagnoses list: all CRITICAL and WARNING diagnoses with full message text
- GC Summary (when a GC log was uploaded): algorithm, GC time%, max pause, P99 pause

12.2 DOCX Report

The DOCX report contains the same content as the PDF but in Microsoft Word format, allowing recipients to edit, annotate, or incorporate it into other documents. Generated using Apache POI OOXML.

12.3 JSON Raw Export

The full analysis result serialized as JSON. Intended for:

- Re-importing into automated tooling
- Custom visualization pipelines
- Archival alongside the original dump files

The JSON includes: the complete call tree, all hotspots, all diagnoses, thread state distribution, timeline data, GC summary, and the health score breakdown.

12.4 CSV Hotspot Export

A flat comma-separated file with one row per hotspot. Column headers:

```
rank,methodSignature,className,packagePrefix,layer,selfTimePercent,totalTimePercent,diagnosis,severity
```

The CSV is compatible with Excel, Google Sheets, and any BI tool that accepts CSV. UTF-8 encoding, no BOM.

12.5 Where to Find Export Buttons

View	Available Exports
Dashboard	PDF, DOCX, JSON, CSV
Hotspot Panel	PDF, DOCX, CSV
Call Tree	PDF, DOCX
Timeline	PDF, DOCX
Diff	PDF, DOCX (based on slot A session)
GC Analyzer	PDF, DOCX

13. Health Score Deep Dive

The Health Score is a single 0-100 numeric indicator that summarizes the overall performance health of a JVM dump. It is designed to give operations teams an instant severity signal without requiring detailed knowledge of thread dump analysis.

13.1 Scoring Formula

The score is computed from four weighted signals. Each signal is normalized to a 0-100 scale, then combined using a weighted average:

Signal	Weight	0 pts (worst)	100 pts (best)
Blocked Threads %	30%	≥ 50% of threads BLOCKED	0% of threads BLOCKED
Confirmed Deadlock	25%	Any confirmed deadlock	No deadlock detected
Critical Diagnoses Count	25%	≥ 5 CRITICAL diagnoses	0 CRITICAL diagnoses
Hotspot Concentration	20%	Top method ≥ 50% Self Time	Top method < 10% Self Time

Each signal score is linearly interpolated between its worst and best values. The final score is the weighted sum, rounded to the nearest integer.

13.2 Classification

Score Range	Classification	Recommended Action
80 - 100	GREEN — Healthy	No immediate action required. Monitor normally.
50 - 79	AMBER — Degraded	Investigate the dominant factor. Schedule a fix.
0 - 49	RED — Critical	Immediate attention required. Escalate to on-call.

13.3 Dominant Factor

The signal that caused the greatest penalty to the score is reported as the Dominant Factor. The penalty for each signal is computed as $(1 - \text{normalizedScore}) * \text{weight}$. The signal with the highest penalty is displayed in the UI below the score, for example:

- "Score reduced primarily by: 38% of threads are BLOCKED" — blocked threads signal is dominant.
- "Score reduced primarily by: 1 confirmed deadlock" — deadlock signal is dominant.
- "Score reduced primarily by: 4 CRITICAL diagnoses" — diagnosis count signal is dominant.

13.4 Configuring Signal Weights

Signal weights can be tuned in the server configuration file (`application.yml`) to match your team's priorities. For example, if deadlock detection is more critical than GC health in your environment, increase the deadlock weight and decrease another weight correspondingly. Weights must sum to 1.0.

```
performanceanalyzer:
  healthscore:
    weights:
      blocked-threads: 0.30
      deadlock: 0.25
      critical-diagnoses: 0.25
```



```
hotspot-concentration: 0.20
```

14. Integration and Metrics

The application exposes integration points for external observability and alerting systems.

14.1 Prometheus Metrics Endpoint

After an analysis is completed, the results are exposed as Prometheus-format gauges at:

```
GET http://localhost:8080/actuator/prometheus
```

Available metrics after analysis:

Metric Name	Description
pa_health_score	Health score of the most recent analysis (0-100)
pa_threads_total	Total thread count
pa_threads_blocked_percent	Percentage of BLOCKED threads
pa_hotspot_top_self_time_percent	Self-time % of the top hotspot (label: method)
pa_diagnoses_critical_count	Number of CRITICAL-severity diagnoses
pa_diagnoses_warning_count	Number of WARNING-severity diagnoses

14.2 JSON Status Endpoint

A JSON status endpoint for lightweight polling or Datadog HTTP checks:

```
GET http://localhost:8080/api/v1/status/latest
```

14.3 Webhook Notifications

An optional webhook can be configured to send a POST request to a URL of your choice after each analysis completes. This enables Slack or PagerDuty alerts via webhook relays.

Configure in application.yml:

```
performanceanalyzer:
  integrations:
    webhook:
      url: "https://hooks.slack.com/services/your-webhook-url"
      api-key: "" # optional bearer token
```

The webhook payload is a fixed JSON schema (v1):

```
{
  "schemaVersion": "1.0",
  "timestamp": "2026-06-14T10:30:00Z",
  "analysisId": "uuid-v4",
  "healthScore": { "value": 42, "classification": "CRITICAL",
    "dominantFactor": "38% of threads are BLOCKED" },
  "summary": { "totalThreads": 250, "blockedThreadsPercent": 38.0,
    "confirmedDeadlock": true, "criticalDiagnosesCount": 3 },
  "dumpTypes": ["THREAD_DUMP", "GC_LOG"]
}
```

Note

Webhook delivery is fire-and-forget with a 5-second timeout. There is no retry on failure. Delivery failures are logged at WARN level on the server but do not affect the analysis response.

14.4 Spring Boot Actuator Endpoints

Endpoint	Description
GET /actuator/health	Spring Boot health check ("status": "UP" when healthy)
GET /actuator/prometheus	Full Prometheus scrape endpoint with all Micrometer gauges
GET /actuator/metrics	Micrometer metrics in JSON format

15. Server Configuration Reference

All server-side configuration is managed in `src/main/resources/application.yml`. Environment variables can override any key using Spring Boot's binding convention (dots replaced by underscores and uppercased).

15.1 Full Configuration File

```
spring:
  servlet:
    multipart:
      max-file-size: 500MB      # Per-file upload limit
      max-request-size: 550MB  # Total request size limit
  mvc:
    async:
      request-timeout: 300000   # 5-minute timeout for large file parsing
  web:
    resources:
      static-locations:
        - classpath:/static/browser/
        - classpath:/static/

server:
  port: 8080

performanceanalyzer:
  session:
    idle-timeout-minutes: 120   # Evict sessions idle for this long

  healthscore:
    weights:
      blocked-threads: 0.30
      deadlock: 0.25
      critical-diagnoses: 0.25
      hotspot-concentration: 0.20

  thresholds:
    min-self-time-critical: 5.0
    blocked-threads-alert: 20.0
    gc-thrash-alert: 10.0
    deadlock-confirmation-threshold: 1.0
    calltree-max-depth: 200

  integrations:
    webhook:
      url: ""                  # Empty = disabled
      api-key: ""
      timeout-ms: 5000

  export:
    pdf:
      top-hotspots-count: 10

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
```

15.2 Environment Variable Overrides

Any configuration property can be overridden via environment variable. Spring Boot converts property paths using the following rule: replace dots (.) with underscores (_), uppercase everything. Examples:

Property	Environment Variable
server.port	SERVER_PORT
performanceanalyzer.session.idle-timeout-minutes	PERFORMANCEANALYZER_SESSION_IDLE_TIMEOUT_MINUTES
performanceanalyzer.integrations.webhook.url	PERFORMANCEANALYZER_INTEGRATIONS_WEBHOOK_URL
spring.servlet.multipart.max-file-size	SPRING_SERVLET_MULTIPART_MAX_FILE_SIZE

15.3 JVM Tuning for Production

Scenario	Recommended Flags
Default workload	-XX:+UseZGC -XX:MaxRAMPercentage=75.0
Large heap dumps (> 200 MB)	Increase container memory limit to 8g
High concurrency (many simultaneous uploads)	Add -Djava.util.concurrent.ForkJoinPool.common.parallelism=4
Debugging startup issues	Add -verbose:class -XX:+PrintCommandLineFlags

16. REST API Reference

All API endpoints are available under the base path `/api/v1/`. All request and response bodies are JSON (Content-Type: `application/json`) unless otherwise noted.

16.1 Endpoint Summary

Method	Endpoint	Description
POST	<code>/api/v1/upload</code>	Upload one or more dump files (multipart/form-data, field name: <code>files</code>)
POST	<code>/api/v1/analyze/{sessionId}</code>	Run full analysis pipeline (body: <code>RulesetDto</code> to JSON)
GET	<code>/api/v1/analyze/{sessionId}</code>	Retrieve cached analysis result
POST	<code>/api/v1/diff</code>	Compare two sessions (<code>?sessionIdA=&sessionIdB=&deltaThreshold=</code>)
GET	<code>/api/v1/export/pdf/{sessionId}</code>	Download PDF report
GET	<code>/api/v1/export/docx/{sessionId}</code>	Download DOCX report
GET	<code>/api/v1/export/json/{sessionId}</code>	Download JSON result
GET	<code>/api/v1/export/csv/{sessionId}/hotspots</code>	Download CSV hotspot list
DELETE	<code>/api/v1/session/{sessionId}</code>	Delete a specific session
DELETE	<code>/api/v1/session/clear</code>	Delete all sessions
GET	<code>/api/v1/status/latest</code>	Latest analysis summary (JSON)
GET	<code>/actuator/health</code>	Spring Boot health check
GET	<code>/actuator/prometheus</code>	Prometheus metrics scrape endpoint

16.2 Upload and Analyze Example (curl)

```
# 1. Upload files
curl -X POST http://localhost:8080/api/v1/upload \
  -F "files=@/path/to/threaddump.txt" \
  -F "files=@/path/to/gc.log"

# Response:
# {
#   "sessionId": "550e8400-e29b-41d4-a716-446655440000",
#   "fileNames": ["threaddump.txt", "gc.log"],
#   "detectedTypes": ["THREAD_DUMP", "GC_LOG"]
# }

# 2. Run analysis (store session ID from previous response)
SESSION_ID="550e8400-e29b-41d4-a716-446655440000"

curl -X POST http://localhost:8080/api/v1/analyze/$SESSION_ID \
  -H "Content-Type: application/json" \
  -d '{"version":"1.0","customSignatures":[],"thresholds":{}}'

# 3. Download PDF
curl -o report.pdf http://localhost:8080/api/v1/export/pdf/$SESSION_ID
```

16.3 Error Response Format

All errors return a structured JSON body:

```
{
  "errorCode": "BAD_REQUEST | NOT_FOUND | FILE_TOO_LARGE | INTERNAL_ERROR",
  "message": "Human-readable description of the error",
  "timestamp": "2026-06-14T10:00:00.000Z"
}
```

17. Troubleshooting

17.1 Application Does Not Start

Symptom	Likely Cause and Fix
"Port 8080 already in use"	Another process is bound to port 8080. Stop the other process, or change the port: <code>java -jar app.jar --server.port=9090</code>
"UnsupportedClassVersionError"	The JAR was compiled with Java 21 but you are running Java 17 or older. Install Java 21+: <code>java -version</code>
JAR file locked (Windows)	The application is still running from a previous build. Stop it (Ctrl+C in the terminal) before running <code>mvn package</code> again.

17.2 Upload Fails

Symptom	Likely Cause and Fix
"File too large" (client-side)	The file exceeds 500 MB. For larger heap dumps, use Eclipse MAT directly.
HTTP 500 after upload	The server ran out of memory during parsing. Increase the container memory limit (<code>mem_limit</code> in <code>docker-compose.yml</code>) or add <code>-Xmx4g</code> to JVM flags.
Detected type is wrong	The format auto-detector may have been confused by an unusual file header. Rename the file to the correct extension (<code>.jfr</code> , <code>.hprof</code> , <code>.log</code>) and re-upload.

17.3 Analysis Shows No Data

Symptom	Likely Cause and Fix
Call Tree is empty after JFR upload	The JFR file was recorded without CPU profiling events. Re-record with the 'profile' settings <code>profile: jcmd <pid> JFR.start settings=profile</code>
GC Analyzer shows no events	The GC log may be in an unexpected format. Check the log file for content and ensure it was generated by <code>-Xlog:gc*</code> or <code>-verbose:gc</code>
Health Score shows 100 with no data	Analysis was not run. Click the Analyze button after upload.
Timeline has only one snapshot	The thread dump file contains only a single snapshot. To get timeline data, generate multiple jstack captures.

17.4 Export Failures

Symptom	Fix
PDF download is a blank file	Re-run the analysis and try exporting again. If the issue persists, check the server log for iText-related errors.
DOCX opens with garbled text	Ensure the Word version supports OOXML (Office 2007 or later). LibreOffice Writer fully supports OOXML.
JSON export is very large (> 50 MB)	Normal for dumps with thousands of threads. Use a streaming JSON viewer such as jq instead of opening in a text editor.

17.5 Browser Issues

Symptom	Fix
Blank page at http://localhost:8080	Clear browser cache and reload. If the issue persists, check the browser console for JavaScript errors.
Charts do not render	Apache ECharts requires a modern browser. Ensure Chrome, Firefox, or Edge is up to date.
Language does not change	Custom settings are stored in localStorage. Ensure localStorage is not blocked by a privacy extension.
Session disappears after tab close and reopen	Browser sessions are NOT lost on tab close — they survive on the server for 120 minutes. If the session list is empty, the 120-minute timeout may have passed or the server was restarted.

18. Appendix

A. Glossary

Term	Definition
BLOCKED	Thread state: the thread is waiting to acquire an object monitor (synchronized lock) held by another thread.
Call Tree	Hierarchical tree of method calls aggregated from all captured stack traces.
CMS	Concurrent Mark Sweep — a deprecated GC algorithm (removed in JDK 15). Replaced by G1GC.
Confirmed Deadlock	A circular wait condition detected when the same threads remain BLOCKED on the same monitors across all snapshots.
Dominant Factor	The signal that caused the greatest reduction in the Health Score.
G1GC	Garbage-First Garbage Collector — default GC since JDK 9. Low-pause, region-based.
Health Score	Composite 0-100 metric computed from blocked threads %, deadlock presence, CRITICAL diagnosis count, and hotspot concentration.
Hotspot	A method with non-zero Self Time %, indicating it was found at the top of a call stack (actually executing) in at least one sample.
JFR	Java Flight Recorder — low-overhead profiling mechanism built into the JDK since Java 11 (preview from 8).
jstack	JDK command-line tool that captures a thread dump of a running JVM process.
Monitor	The synchronization primitive associated with every Java object. A thread acquires a monitor when entering a synchronized block or method.
RUNNABLE	Thread state: the thread is executing or ready to execute on a CPU.
Self Time %	Percentage of stack samples where this method was the top-of-stack (innermost) frame.
STW	Stop-The-World — a GC pause during which all application threads are suspended.
TIMED_WAITING	Thread state: the thread is waiting with a timeout (Thread.sleep, Object.wait(timeout), LockSupport.parkNanos).
Total Time %	Percentage of stack samples where this method appeared anywhere in the call stack.
WAITING	Thread state: the thread is waiting indefinitely (Object.wait(), LockSupport.park).
ZGC	Z Garbage Collector — sub-millisecond pause GC available since JDK 11 (production-ready JDK 15+).

B. Keyboard Shortcuts

The application does not define custom keyboard shortcuts. Standard browser shortcuts apply (Ctrl+R to refresh, Ctrl+F to find in page, etc.).

C. Browser LocalStorage Keys

Key	Purpose	Values
pa.theme	Display theme preference	"light" or "dark"
pa.lang	Interface language preference	"en", "es", "fr", "de", "pt", "it"
pa.ruleset	Custom signatures and thresholds	JSON object (see Section 11.5)

D. Diagnostic Rule Reference

The following built-in rules are applied on every analysis:

Rule ID	Severity	Trigger Condition
BLOCKED_THREADS	WARNING / CRITICAL	More than 20% of threads in BLOCKED state (threshold configurable)
DEADLOCK	CRITICAL	Circular dependency detected in the lock graph
LIST_LINEAR_SCAN	WARNING	High Self Time on ArrayList.indexOf, ArrayList.contains, or LinkedList traversal
STRING_CONCAT_LOOP	WARNING	High Self Time on StringBuilder.append or String.concat
REFLECTION_ABUSE	WARNING	High Self Time on java.lang.reflect.Method.invoke
FINALIZER_BACKLOG	WARNING	java.lang.ref.Finalizer thread permanently RUNNABLE at top of stack
GC_THRASHING	WARNING / CRITICAL	GC time exceeds configured threshold (default 10%)

E. Release Notes — Version 1.0 (June 2026)

- Initial release with support for thread dumps, JFR recordings (CPU profiling), GC logs (unified and legacy), and heap dump storage.
- Health Score engine with four configurable signals.
- Diff engine for side-by-side session comparison.
- Timeline analyzer for multi-snapshot dumps.
- Export in PDF (iText 8), DOCX (Apache POI), JSON, and CSV.
- Multilingual UI: English, Spanish, French, German, Portuguese, Italian.
- Dark mode with localStorage persistence.
- Custom rules engine with ruleset import/export.
- Prometheus and JSON metrics endpoints.
- Optional webhook notification after analysis completion.
- Fat JAR packaging — single self-contained executable with embedded Angular SPA.

Known limitations in v1.0:

- Heap dump analysis requires Eclipse MAT as a standalone tool (Eclipse MAT API not on Maven Central).
- No user authentication — designed for local or internal-network deployment only.
- JFR allocation profiling events (jdk.ObjectAllocationInNewTLAB) are not yet parsed.
- Mobile viewport not supported (minimum 1280 px).